Вариант №22

Цель работы: изучить основные понятия теории формальных языков и грамматик.

1. Найти терминальную цепочку α, $|\alpha| > 10$, для которой существует не менее двух левых выводов в заданной КС-грамматике (см. варианты заданий). Записать различные левые выводы этой цепочки. Построить деревья вывода. Определить последовательности правил, применяемые при этих выводах.

2. Написать программу, которая определяет, можно ли применить заданную последовательность правил при левом выводе терминальной цепочки в заданной КС-грамматике, формирует левый вывод и линейную скобочную форму дерева вывода. Обработать программой последовательности правил, полученные в п.1. Примечание. Если к нетерминалу A в процессе вывода применяется правило с номером n, то в выводе и в линейной скобочной форме дерева вывода после нетерминала A должен быть символ с кодом n.

3. Найти последовательность правил p, $|p| > 10$, которую можно применить при произвольном выводе терминальной цепочки, но нельзя применить при левом или правом выводе в заданной КС-грамматике (см. варианты заданий). Записать вывод v, в процессе которого применяется последовательность правил p. Построить дерево вывода. Записать левый и правый выводы, эквивалентные выводу v.

4. Написать программу, которая определяет, можно ли применить заданную последовательность правил p при выводе терминальной цепочки в заданной КС-грамматике и формирует линейную скобочную форму дерева вывода. Если последовательность правил p можно применить при выводе v терминальной цепочки, то программа должна вывести последовательность правил, применяемую при левом выводе, эквивалентном выводу v. Обработать программой последовательность правил, найденную в п.3. Примечание. Если к нетерминалу A в процессе вывода применяется правило с номером n, то в линейной скобочной форме дерева вывода после нетерминала A должен быть символ с кодом n.

Задание №1 Найти терминальную цепочку α, $|\alpha| > 10$, для которой существует не менее двух левых выводов в заданной КС-грамматике (см. варианты заданий). Записать различные левые выводы этой цепочки. Построить деревья вывода. Определить последовательности правил, применяемые при этих выводах.

Правила:

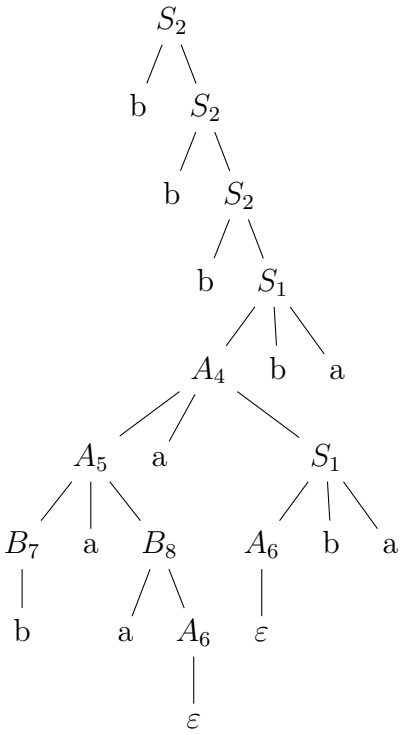
1.	$S \to Aba$	5.	$A \to BaB$
2.	$S \to bS$	6.	$A \to \varepsilon$
3.	$S \to cAbA$	7.	$B \to b$
4.	$A \to AaS$	8.	$B \to aA$

1. Первый левый вывод:

$$S_2 \Rightarrow bS_2 \Rightarrow bbS_2 \Rightarrow bbbS_1 \Rightarrow bbbA_4ba \Rightarrow bbbA_5aSba \Rightarrow bbbB_7aBaSba$$
$$\Rightarrow bbbbaB_8aSba \Rightarrow bbbbaaA_6aSba \Rightarrow bbbbaaaS_1ba$$
$$\Rightarrow bbbbaaaA_6baba \Rightarrow bbbbaaababa$$

Последовательность правил: $2, 2, 2, 1, 4, 5, 7, 8, 6, 1, 6$.

2. Дерево вывода для первого левого вывода

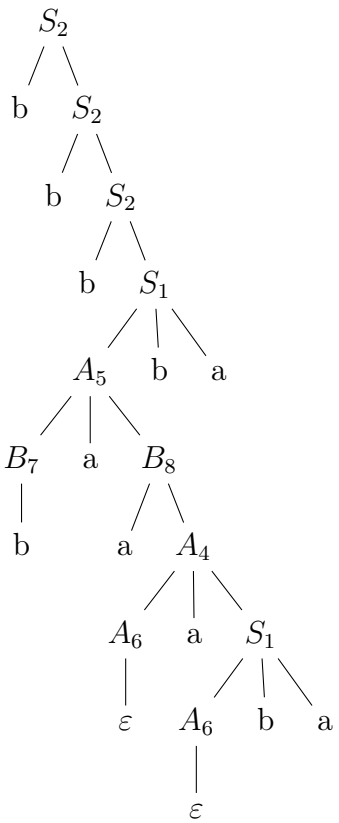


3. Второй левый вывод:

$$S_2 \Rightarrow bS_2 \Rightarrow bbS_2 \Rightarrow bbbS_1 \Rightarrow bbbA_5ba \Rightarrow bbbB_7aBba \Rightarrow bbbbaB_8ba$$
$$\Rightarrow bbbbaaA_4ba \Rightarrow bbbbaaA_6aSba \Rightarrow bbbbaaaS_1ba$$
$$\Rightarrow bbbbaaaA_6baba \Rightarrow bbbbaaababa$$

Последовательность правил: $2, 2, 2, 1, 5, 7, 8, 4, 6, 1, 6$.

4. Дерево вывода для второго левого вывода



Задание №2 Написать программу, которая определяет, можно ли применить заданную последовательность правил при левом выводе терминальной цепочки в заданной КС-грамматике, формирует левый вывод и линейную скобочную форму дерева вывода. Обработать программой последовательности правил, полученные в п.1. Примечание. Если к нетерминалу A в процессе вывода применяется правило с номером n, то в выводе и в линейной скобочной форме дерева вывода после нетерминала A должен быть символ с кодом n.

```cpp
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

const char lhs[9]   = {' ', 'S', 'S', 'S', 'A', 'A', 'A', 'B', 'B'};
const string rhs[9] = {"", "Aba", "bS", "cAbA", "AaS", "BaB", "e", "b", "aA"};

bool isNonterminal(char c)
{
    return isupper(static_cast<unsigned char>(c)) != 0;
}
```

```
13
14  string applyRule(const string& form, int pos, int rule)
15  {
16      return form.substr(0, pos) + rhs[rule] + form.substr(pos + 1);
17  }
18
19  int findLeftmostNonterminal(const string& form)
20  {
21      for (int i = 0; i < (int)form.size(); ++i)
22          if (isNonterminal(form[i]))
23              return i;
24      return -1;
25  }
26
27  int findFirstStar(const string& lsf)
28  {
29      size_t pos = lsf.find('*');
30      if (pos == string::npos)
31          return -1;
32      return (int)pos;
33  }
34
35  string expandRuleForLsf(char nonTerm, int rule)
36  {
37      string repl = string(1, nonTerm) + "(" + to_string(rule) + ")(";
38      const string& r = rhs[rule];
39
40      for (int i = 0; i < (int)r.size(); ++i)
41      {
42          if (i > 0)
43              repl += " ";
44
45          char c = r[i];
46
47          if (c == 'e')
48              repl += "";
49          else if (isNonterminal(c))
50          {
51              repl += '*';
52              repl += c;
53          }
54          else
55              repl += c;
```

```
56      }
57
58      repl += ")";
59      return repl;
60  }
61
62  string updateLsf(const string& lsf, char nonTerm, int rule)
63  {
64      int starPos = findFirstStar(lsf);
65      if (starPos < 0)
66          return lsf;
67
68      string repl = expandRuleForLsf(nonTerm, rule);
69
70      return lsf.substr(0, starPos) + repl + lsf.substr(starPos + 2);
71  }
72
73  void printSequence(const int* seq, int n)
74  {
75      cout << "Последовательность: ";
76      for (int i = 0; i < n; ++i)
77          cout << seq[i] << " ";
78  }
79
80  void printStep(const string& form, int pos, int rule)
81  {
82      cout << " => ";
83
84      for (int i = 0; i < (int)form.size(); ++i)
85      {
86          if (i == pos)
87              cout << form[i] << "(" << rule << ")";
88          else if (form[i] == 'e')
89              cout << "";
90          else
91              cout << form[i];
92      }
93  }
94
95  bool hasNonterminals(const string& form)
96  {
97      for (char c : form)
98          if (isNonterminal(c))
```

```
99              return true;
100     return false;
101 }
102
103 void printFinal(const string& form, const string& lsf)
104 {
105     cout << "\nЦепочка: ";
106     for (char c : form)
107         if (c != 'e')
108             cout << c;
109
110     cout << "\nЛСФ: " << lsf << "\n";
111 }
112
113 int process(const int* seq, int n)
114 {
115     string form = "S";
116     string lsf  = "*S";
117
118     printSequence(seq, n);
119     cout << "\nS";
120
121     for (int step = 0; step < n; ++step)
122     {
123         int rule = seq[step];
124
125         int pos = findLeftmostNonterminal(form);
126         if (pos < 0)
127         {
128             cout << "\nОшибка: нет нетерминалов\n";
129             return 0;
130         }
131
132         if (form[pos] != lhs[rule])
133         {
134             cout << "\nОшибка: правило " << rule
135                  << " не применимо к " << form[pos] << "\n";
136             return 0;
137         }
138
139         printStep(form, pos, rule);
140         lsf  = updateLsf(lsf, form[pos], rule);
141         form = applyRule(form, pos, rule);
```

```
142     }
143
144     if (hasNonterminals(form))
145     {
146         cout << "\nОшибка: остались нетерминалы\n";
147         return 0;
148     }
149
150     printFinal(form, lsf);
151     return 1;
152 }
153
154 int main()
155 {
156     int seq1[] = {2, 2, 2, 1, 4, 5, 7, 8, 6, 1, 6};
157     int seq2[] = {2, 2, 2, 1, 5, 7, 8, 4, 6, 1, 6};
158
159     cout << "Последовательность 1\n";
160     process(seq1, 11);
161
162     cout << "\nПоследовательность 2\n";
163     process(seq2, 11);
164
165     return 0;
166 }
```

```
Последовательность 1
Последовательность: 2 2 2 1 4 5 7 8 6 1 6
S => S(2) => bS(2) => bbS(2) => bbbS(1) => bbbA(4)ba => bbbA(5)aSba => bbbB(7)aBaSba => bbbbaB(8)aSba
=> bbbbaaA(6)aSba => bbbbaaεaS(1)ba => bbbbaaεaA(6)baba
Цепочка: bbbbaaababa
ЛСФ: S(2)(b S(2)(b S(2)(b S(1)(A(4)(A(5)(B(7)(b) a B(8)(a A(6)(ε))) a S(1)(A(6)(ε) b a)) b a))))

Последовательность 2
Последовательность: 2 2 2 1 5 7 8 4 6 1 6
S => S(2) => bS(2) => bbS(2) => bbbS(1) => bbbA(5)ba => bbbB(7)aBba => bbbbaB(8)ba => bbbbaaA(4)ba =>
bbbbaaA(6)aSba => bbbbaaεaS(1)ba => bbbbaaεaA(6)baba
Цепочка: bbbbaaababa
ЛСФ: S(2)(b S(2)(b S(2)(b S(1)(A(5)(B(7)(b) a B(8)(a A(4)(A(6)(ε) a S(1)(A(6)(ε) b a)))) b a))))
```

Рис. 1: Результат выполнения программ

Задание №3 Найти последовательность правил p, $|p| > 10$, которую можно применить при произвольном выводе терминальной цепочки, но нельзя применить при левом или правом выводе в заданной КС-грамматике (см. варианты заданий). Записать вывод v, в процессе которого применяется последовательность правил p. Построить дерево вывода. Записать левый и правый выводы, эквивалентные выводу v.

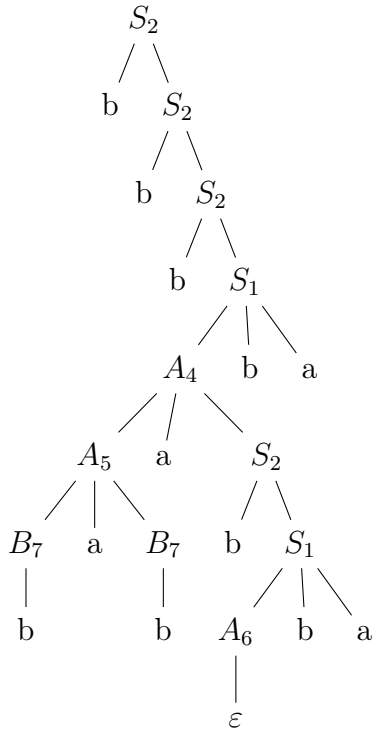
1. Произвольный вывод v, в котором применяется p

1.	$S \to Aba$	5.	$A \to BaB$
2.	$S \to bS$	6.	$A \to \varepsilon$
3.	$S \to cAbA$	7.	$B \to b$
4.	$A \to AaS$	8.	$B \to aA$

$$
\begin{aligned}
S_2 &\Rightarrow bS_2 \Rightarrow bbS_2 \Rightarrow bbbS_1 \Rightarrow bbbA_4ba \Rightarrow bbbA_5aSba \\
&\Rightarrow bbbBaBaS_2ba \Rightarrow bbbBaB_7abSba \Rightarrow bbbB_7ababSba \\
&\Rightarrow bbbbababS_1ba \Rightarrow bbbbababA_6baba \Rightarrow bbbbababbaba
\end{aligned}
$$

Последовательность правил: $2, 2, 2, 1, 4, 5, 2, 7, 7, 1, 6$.

2. Дерево вывода



3. Левый вывод $2, 2, 2, 1, 4, 5$

$$S_2 \Rightarrow bS_2 \Rightarrow bbS_2 \Rightarrow bbbS_1 \Rightarrow bbbA_4ba \Rightarrow bbbA_5aSba \Rightarrow bbbBaBaS_2ba$$

Крайний слева нетерминал — это B. Следующее правило в p — 2 ($S \to bS$), но его нельзя применить к B. Значит, левый вывод по этой последовательности невозможен.

4. Правый вывод $2, 2, 2, 1, 4$

$$S_2 \Rightarrow bS_2 \Rightarrow bbS_2 \Rightarrow bbbS_1 \Rightarrow bbbA_4ba \Rightarrow bbbA_5aSba$$

Крайний справа нетерминал — это S. Следующее правило в p — 5 ($A \to BaB$), но его нельзя применить к S. Значит, правый вывод по этой последовательности тоже невозможен.

5. Эквивалентные левый и правый выводы с индексами правил

Ту же терминальную цепочку

$$w = bbbbababbaba$$

можно получить левым и правым выводами, но с другим порядком применения правил.

(a) Левый вывод

$$\begin{aligned} S_2 &\Rightarrow bS_2 \Rightarrow bbS_2 \Rightarrow bbbS_1 \Rightarrow bbbA_4ba \\ &\Rightarrow bbbA_5aSba \Rightarrow bbbB_7aBaSba \\ &\Rightarrow bbbbaB_7aSba \Rightarrow bbbbaaS_2ba \\ &\Rightarrow bbbbaabS_1ba \Rightarrow bbbbaabA_6baba \\ &\Rightarrow bbbbaababbaba \end{aligned}$$

Последовательность правил левого вывода:

$$2,\ 2,\ 2,\ 1,\ 4,\ 5,\ 7,\ 7,\ 2,\ 1,\ 6.$$

(b) Правый вывод

$$\begin{aligned} S_2 &\Rightarrow bS_2 \Rightarrow bbS_2 \Rightarrow bbbS_1 \Rightarrow bbbA_4ba \\ &\Rightarrow bbbAaS_2ba \Rightarrow bbbAabS_1ba \Rightarrow bbbAabA_6baba \\ &\Rightarrow bbbA_5abbaba \Rightarrow bbbBaB_7abbaba \\ &\Rightarrow bbbB_7ababbaba \Rightarrow bbbbababbaba \end{aligned}$$

Последовательность правил правого вывода:

$$2,\ 2,\ 2,\ 1,\ 4,\ 2,\ 1,\ 6,\ 5,\ 7,\ 7.$$

Задание №4 Написать программу, которая определяет, можно ли применить заданную последовательность правил p при выводе терминальной цепочки в заданной КС-грамматике и формирует линейную скобочную форму дерева вывода. Если последовательность правил p можно применить при выводе v терминальной цепочки, то программа должна вывести последовательность правил, применяемую при левом выводе, эквивалентном выводу v. Обработать программой последовательность правил, найденную в п.3. Примечание. Если к нетерминалу A в процессе вывода применяется правило с номером n, то в линейной скобочной форме дерева вывода после нетерминала A должен быть символ с кодом n.

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <cctype>
using namespace std;

const char lhs[9]   = {' ', 'S', 'S', 'S', 'A', 'A', 'A', 'B', 'B'};
const string rhs[9] = {"", "Aba", "bS", "cAbA", "AaS", "BaB", "e", "b", "aA"};

bool isNonterminal(char c)
{
    return isupper(static_cast<unsigned char>(c)) != 0;
}

bool isDigitChar(char c)
{
    return isdigit(static_cast<unsigned char>(c)) != 0;
}

string applyRule(const string& form, int pos, int rule)
{
    return form.substr(0, pos) + rhs[rule] + form.substr(pos + 1);
}

int countNonterminalsBefore(const string& form, int pos)
{
    int cnt = 0;
    for (int i = 0; i < pos && i < (int)form.size(); ++i)
        if (isNonterminal(form[i]))
            ++cnt;
    return cnt;
}

int findKthStar(const string& lsf, int k)
{
    int cnt = 0;
    for (int i = 0; i < (int)lsf.size(); ++i)
    {
        if (lsf[i] == '*')
        {
            if (cnt == k)
                return i;
```

```
            ++cnt;
        }
    }
    return -1;
}

string expandRuleForLsf(char nonTerm, int rule)
{
    string repl = string(1, nonTerm) + "(" + to_string(rule) + ")(";
    const string& r = rhs[rule];

    for (int i = 0; i < (int)r.size(); ++i)
    {
        if (i > 0)
            repl += " ";

        char c = r[i];

        if (c == 'e')
            repl += "";
        else if (isNonterminal(c))
        {
            repl += '*';
            repl += c;
        }
        else
            repl += c;
    }

    repl += ")";
    return repl;
}

string updateLsf(const string& lsf, int k, char nonTerm, int rule)
{
    int starPos = findKthStar(lsf, k);
    if (starPos < 0)
        return lsf;

    return lsf.substr(0, starPos) + expandRuleForLsf(nonTerm, rule)
           + lsf.substr(starPos + 2);
}

```

```
86  string cleanLsf(const string& lsf)
87  {
88      string s;
89      for (char c : lsf)
90          if (c != '*')
91              s += c;
92      return s;
93  }
94
95  bool hasNonterminals(const string& form)
96  {
97      for (char c : form)
98          if (isNonterminal(c))
99              return true;
100     return false;
101 }
102
103 void printRuleSequence(const vector<int>& p)
104 {
105     cout << "Последовательность правил p: ";
106     for (int r : p)
107         cout << r << " ";
108 }
109
110 bool search(const vector<int>& p, int step, const string& form, vector<int>& positions)
111 {
112     if (step == (int)p.size())
113         return !hasNonterminals(form);
114
115     char need = lhs[p[step]];
116
117     for (int i = 0; i < (int)form.size(); ++i)
118     {
119         if (form[i] != need)
120             continue;
121
122         string next = applyRule(form, i, p[step]);
123         positions.push_back(i);
124
125         if (search(p, step + 1, next, positions))
126             return true;
127
128         positions.pop_back();
```

```
    }

    return false;
}

string buildLsf(const vector<int>& p, const vector<int>& positions)
{
    string form = "S";
    string lsf  = "*S";

    for (int step = 0; step < (int)p.size(); ++step)
    {
        int rule = p[step];
        int pos  = positions[step];
        int k    = countNonterminalsBefore(form, pos);
        char nonTerm = form[pos];

        lsf  = updateLsf(lsf, k, nonTerm, rule);
        form = applyRule(form, pos, rule);
    }

    return lsf;
}

string getTerminalChain(const vector<int>& p, const vector<int>& positions)
{
    string form = "S";

    for (int step = 0; step < (int)p.size(); ++step)
        form = applyRule(form, positions[step], p[step]);

    string result;
    for (char c : form)
        if (c != 'e')
            result += c;

    return result;
}

bool matchRuleAt(const string& lsf, int i, int& rule, int& nextPos)
{
    if (!isNonterminal(lsf[i]))
        return false;
```

```
172
173     if (i + 1 >= (int)lsf.size() || lsf[i + 1] != '(')
174         return false;
175
176     int j = i + 2;
177     int r = 0;
178
179     while (j < (int)lsf.size() && isDigitChar(lsf[j]))
180     {
181         r = r * 10 + (lsf[j] - '0');
182         ++j;
183     }
184
185     if (j >= (int)lsf.size() || lsf[j] != ')')
186         return false;
187
188     rule = r;
189     nextPos = j + 1;
190     return true;
191 }
192
193 vector<int> extractLeftmost(const string& lsf)
194 {
195     vector<int> seq;
196
197     for (int i = 0; i < (int)lsf.size(); ++i)
198     {
199         int rule = 0;
200         int nextPos = i;
201
202         if (matchRuleAt(lsf, i, rule, nextPos))
203         {
204             seq.push_back(rule);
205             i = nextPos - 1;
206         }
207     }
208
209     return seq;
210 }
211
212 int main()
213 {
214     vector<int> p = {2, 2, 2, 1, 4, 2, 5, 7, 7, 1, 6};
```

```
215
216     printRuleSequence(p);
217     cout << "\n";
218
219     vector<int> positions;
220
221     if (search(p, 0, "S", positions))
222     {
223         cout << "Последовательность применима при произвольном выводе.\n";
224
225         string lsf = buildLsf(p, positions);
226         cout << "ЛСФ дерева вывода:\n" << cleanLsf(lsf) << "\n";
227
228         vector<int> leftSeq = extractLeftmost(lsf);
229         cout << "Последовательность правил при левом выводе, "
230                 "эквивалентном v: ";
231         for (int r : leftSeq)
232             cout << r << " ";
233         cout << "\n";
234
235         cout << "Терминальная цепочка: "
236              << getTerminalChain(p, positions) << "\n";
237     }
238     else
239     {
240         cout << "Последовательность не применима "
241                 "при произвольном выводе.\n";
242     }
243
244     return 0;
245 }
```

```
Последовательность правил p: 2 2 2 1 4 5 2 7 7 1 6
Последовательность применима при произвольном выводе.
ЛСФ дерева вывода:
S(2)(b S(2)(b S(2)(b S(1)(A(4)(A(5)(B(7)(b) a B(7)(b)) a S(2)(b S(1)(A(6)(ε) b a))) b a))))
Последовательность правил при левом выводе, эквивалентном v: 2 2 2 1 4 5 7 7 2 1 6
Терминальная цепочка: bbbbababbaba
```

Рис. 2: Результат выполнения программ